Big Data Analytics for LHC Event Quality and Exploratory Outlier Ranking in 40 MHz Training Data

Big Data Analytics CA-3 Case Study Report

Name:	Soham Kulkarni	**PRN:**	23070126129
Course:	Big Data Analytics (CA3)	**Division:**	AIML B2
Batch:	Final Year AIML B2, 2023–27	**Platform:**	Hadoop / Hive / Spark / Pig

1 Problem Statement

The 40 MHz benchmark represents reconstructed trigger-level objects from simulated proton-proton collisions. Its tensor layout is compact at the file level, while the raw numerical array has a substantially larger uncompressed representation. Reliable analysis requires explicit handling of tensor dimensions, data types, structural padding, and event-level feature definitions before SQL or machine-learning operations begin.

The research question is: *Can a reproducible, bounded-memory pipeline validate every event in the published training file, transform its object tensor into an auditable analytical table, and use Big Data operations to describe event kinematics and rank unusual feature combinations on a six-core Linux host?*

The dataset file `background_for_training.h5` (646.4 MB, MD5 `08cb4801229f9d480567f8d3f5cac134`) from Zenodo Record 5046428 contains 4,000,000 collision events according to the dataset paper [1]. Each event is a (19×4) floating-point tensor with 304,000,000 total scalar fields.

Scope Boundaries: Spark runs in `local[6]` mode inside one Docker host. This configuration exercises parallel tasks and shuffle behaviour within one machine. It does not measure multi-node speed-up or Level-1 trigger latency. The primary training record supplies no per-event label identifying a hidden new-physics signal; the project therefore does not calculate supervised classification accuracy, ROC-AUC, signal efficiency, precision, recall, or discovery significance from this file alone.

2 Objectives

1. Verify the source record, file size, source-listed checksum, HDF5 keys, tensor dimensions, data type, and field metadata.
2. Scan every tensor chunk and count non-finite values, invalid particle codes, non-integral type codes, negative occupied-object transverse momentum, out-of-range azimuths, slot-type mismatches, and inconsistent padding.
3. Transform each source event into one typed Parquet row containing physics summaries and explicit quality indicators. Preserve the source-row ordinal and prove row-count parity.
4. Quantify electron, muon, and jet multiplicities; summarize the missing-transverse-momentum magnitude and scalar jet and lepton activity; inspect conditional distributions and correlations.
5. Run documented Spark SQL queries and a Spark RDD key-value reduction over the complete transformed table. Maintain a count-parity assertion between event totals and grouped totals.
6. Provide Hadoop Streaming, Hive DDL, and Pig Latin artefacts with their execution requirements documented accurately.

7. Optionally fit a streaming standardizer and MiniBatchKMeans model, then calculate nearest-centroid distances for every event. Interpret scores as exploratory geometric ranks only.
8. Produce figures whose axes, units, aggregation basis, and scientific question are stated in captions. Store machine-readable run manifests for reproducibility.

3 Background and Physics Context

3.1 Trigger-level data analytics

At the Large Hadron Collider, proton bunch crossings occur at a nominal frequency of 40 MHz, corresponding to 25 ns between crossings. Online trigger systems reduce the event rate before permanent storage. Their algorithms use reconstructed objects such as electrons, muons, jets, and missing transverse momentum. The open benchmark supports research on anomaly-detection methods operating in a trigger-level setting [1, 2, 3].

The data product analysed here is an offline file released for research. Wall-clock timings produced by this project describe file processing on the chosen host. They do not establish that a model could make a trigger decision within a hardware latency budget.

3.2 Simulation and event selection

The dataset paper documents simulated proton-proton collisions at a centre-of-mass energy of 13 TeV, generated with PYTHIA and passed through a parametric detector simulation based on DELPHES. The training sample uses a single-lepton trigger selection. The published selection requires an electron or muon above a transverse-momentum threshold, with species-dependent pseudorapidity acceptance. Reconstructed objects are represented by a bounded number of slots per event [1].

Observed distributions reflect the generator configuration, detector simulation, object reconstruction, and event selection. They are not an unbiased census of every physical collision at the LHC.

3.3 Standard Model Process Mixture

The background training corpus represents a physics cocktail of four dominant Standard Model processes produced in high-energy pp collisions [1]:

Physics Process	**Fraction (%)**	**Signature**
$W \to \ell\nu$ (Inclusive W boson)	59.2%	High-p_T isolated lepton recoiling against large missing transverse energy from the undetected neutrino; typically low jet multiplicity.
QCD Multijet Production	33.8%	High hadronic jet activity with leptons from heavy-flavor decays or jet misidentification; low true MET.
$Z \to \ell\ell$ (Inclusive Z boson)	6.7%	Pair of oppositely charged same-flavor leptons with invariant mass near $m_Z \approx 91.2$ GeV; low MET.
$t\bar{t}$ (Top-quark pair production)	0.3%	Complex topologies containing multiple b-jets, light jets, isolated leptons, and MET from W decays; very high scalar energy scale.

The paper describes the Standard Model training cocktail with approximate generator-process proportions. These are published sample-level proportions. The fourth tensor component encodes reconstructed object type; it does not encode the underlying event's generator process. The main training file cannot reconstruct process-mixture percentages from the type-code field. The integer type codes stored in the dataset indicate reconstructed particle species (MET, electron, muon, jet, padding), not generator process classes. No per-event process truth is available in the training corpus.

3.4 Physics terms required to read the analysis

A collision event is represented by reconstructed physics objects. An object slot stores transverse momentum p_{T}, pseudorapidity η, azimuth ϕ, and a categorical type code. Transverse momentum is the magnitude of momentum projected onto the plane perpendicular to the beam axis. For Cartesian components p_x, p_y,

$$p_{\mathrm{T}} = \sqrt{p_x^2 + p_y^2}. \tag{1}$$

In the dataset convention, transverse momentum is expressed in GeV under natural units with $c = 1$.

Pseudorapidity is defined from polar angle θ, measured relative to the beam direction:

$$\eta = -\ln\left[\tan\left(\frac{\theta}{2}\right)\right]. \tag{2}$$

For highly relativistic particles, pseudorapidity approximates rapidity, ensuring differences $\Delta\eta$ are invariant under longitudinal Lorentz boosts.

The variable provides a convenient angular coordinate for collider detectors. Azimuth ϕ specifies orientation in the transverse plane and is periodic over an interval represented as $[-\pi, \pi]$. An angular difference must respect this periodic boundary:

$$\Delta\phi(a, b) = \operatorname{atan2}\big(\sin(\phi_a - \phi_b), \cos(\phi_a - \phi_b)\big). \tag{3}$$

The current feature table retains the stored MET azimuth but does not implement pairwise angular separation as a derived feature.

Missing transverse momentum is a vector quantity representing transverse momentum imbalance. The table uses the magnitude stored in the designated MET slot. The pipeline does not reconstruct the vector from the other slots, so interpretations remain tied to the published object representation.

Jet multiplicity N_{jets} counts reconstructed jet slots. Scalar jet activity H_T sums jet transverse-momentum magnitudes. Scalar lepton activity S_T^{ℓ} sums electron and muon transverse-momentum magnitudes. These scalar quantities describe event activity and support stratified summaries. They are not invariant-mass observables.

3.5 Fixed-Width Tensor Layout and Padding Semantics

Each event is a fixed (19×4) tensor. The 19 slots are allocated into four object segments:

$$\text{Slot Layout:}\quad \underbrace{\text{Slot } 0}_{\text{MET } (t=1)} \quad \underbrace{\text{Slots } 1\ldots4}_{\text{Electrons } (t=2)} \quad \underbrace{\text{Slots } 5\ldots8}_{\text{Muons } (t=3)} \quad \underbrace{\text{Slots } 9\ldots18}_{\text{Jets } (t=4)}. \tag{4}$$

Within each group, objects are sorted in descending transverse momentum. Unused slots have type code $t_i = 0$ and all kinematic fields set to 0.0.

Type code 0 denotes structural padding required by the tensor shape. It is not missing data or a corrupt reading. The ETL pipeline distinguishes valid padding from corruption by checking that padding slots have all zero kinematic fields:

$$(t_i = 0) \implies (p_{T,i} = 0.0 \wedge \phi_i = 0.0 \wedge \eta_i = 0.0). \tag{5}$$

3.6 Unsupervised anomaly scoring rationale

Searching for unexpected new physics without specific model assumptions requires unsupervised anomaly detection. Anomalous events are rare and unpredictable, so supervised classifiers cannot be trained against known signal topologies.

This study implements an unsupervised clustering model (`MiniBatchKMeans`, $K = 8$) operating on standardized kinematic features $\mathbf{z} \in \mathbb{R}^{10}$. The atypicality score $s(\mathbf{x})$ of an event $\mathbf{x}$ is the minimum Euclidean distance in standardized feature space to the nearest cluster centroid $\boldsymbol{\mu}_k$:

$$s(\mathbf{x}) = \min_k \|\mathbf{z}(\mathbf{x}) - \boldsymbol{\mu}_k\|_2. \tag{6}$$

This distance measures geometric eccentricity relative to typical Standard Model distributions. Events with large $s(\mathbf{x})$ reside in sparse topological regions. The score is an exploratory distance metric, not a calibrated probability of new physics [3].

4 Dataset Description

4.1 Source record and provenance

Table 1: Source-level dataset contract. The local audit manifest must confirm actual file properties before result generation.

Property	Specification
Repository	Zenodo record 5046428, DOI 10.5281/zenodo.5046428.
File name	`background_for_training.h5`.
Reported file size	Approximately 646.4 MB on the record listing; the fetch manifest records actual bytes.
Source-listed MD5	`08cb4801229f9d480567f8d3f5cac134`.
Container format	HDF5, a hierarchical container for groups, datasets, metadata, and attributes.
Primary key	`/Particles`.
Metadata keys	`/Particles_Names`, `/Particles_Classes`.
Published training cardinality	4,000,000 events according to the dataset article's data-record table.
Expected object tensor shape	$(N, 19, 4)$; checksum-verified local data supplies the authoritative runtime shape.

The source paper lists 4,000,000 events for the Standard Model training record and additional, separately released records for four signal hypotheses and a mixed black-box sample [1]. This project includes the complete main training file. It excludes the separate signal and black-box records. No concatenation or synthetic row expansion is performed.

A source-file byte count and checksum establish which artifact was processed. The MD5 value above is the checksum listed by the repository for download verification. The run manifest also records SHA-256 for local traceability; that digest must be read from the generated manifest and should not be typed into the report before successful local verification.

4.2 Tensor axes and stored fields

The published tensor schema is a three-dimensional array with dimensions $(N, 19, 4)$. Axis 0 indexes events, axis 1 indexes fixed object slots, and axis 2 indexes four values per slot. The expected slot layout is shown in Table 2.

Table 2: Object-slot layout documented for the 40 MHz dataset [1].

Slot indices	Purpose	Type code	Interpretation
0	MET object	1	Stored missing-transverse-momentum magnitude and azimuth.
1–4	Electrons	2	Up to four reconstructed electron objects.
5–8	Muons	3	Up to four reconstructed muon objects.
9–18	Jets	4	Up to ten reconstructed jets.
Unused positions	Padding	0	Empty slots represented with zero-valued fields.

The final tensor component carries the categorical code. Because HDF5 may store the complete tensor in a floating-point dtype, the implementation checks that each finite type value lies within tolerance of an integer and belongs to $\{0, 1, 2, 3, 4\}$ before casting. A direct integer cast without this check could silently map malformed values onto valid category codes.

For the published shape $(4{,}000{,}000, 19, 4)$, the tensor contains

$$4{,}000{,}000 \times 19 \times 4 = 304{,}000{,}000 \tag{7}$$

scalar entries. For a verified `float64` dtype, the uncompressed numeric payload is $304{,}000{,}000 \times 8 = 2.432 \times 10^9$ bytes, before HDF5 chunk metadata and compression. This calculation follows from published cardinality; the audit records the observed runtime shape and dtype.

4.3 Structural padding and data quality

Each event has fixed slot capacity. A zero type code denotes padding when all three kinematic coordinates in that slot are also zero. Padding is an expected representation detail. It must be counted separately from invalid numeric values, invalid type codes, and negative transverse momentum in occupied slots.

The audit implements these checks:

- finite scalar check for every tensor value, with separate NaN and positive/negative infinity counts;
- categorical type-code check, including non-integral values and unknown codes;
- expected slot-position check for occupied MET, electron, muon, and jet objects;
- negative p_{T} count among occupied objects;
- azimuth-boundary count for occupied objects under a documented numerical tolerance;
- padding-consistency check for empty slots containing nonzero kinematic values;
- slot-occupancy and per-event multiplicity distributions.

Execution results are generated from the actual audit manifest. Zero defect counts are conclusions only after the complete file has been scanned successfully.

4.4 Limitations of the record

The main file is simulated and selected. It has no documented event-level truth field for the hidden black-box signal. Object type codes identify reconstructed object categories. The training file cannot establish signal discrimination quality in isolation. Detector response, generator configuration, trigger selection, pile-up settings, and fixed object limits shape feature distributions and must remain part of interpretation.

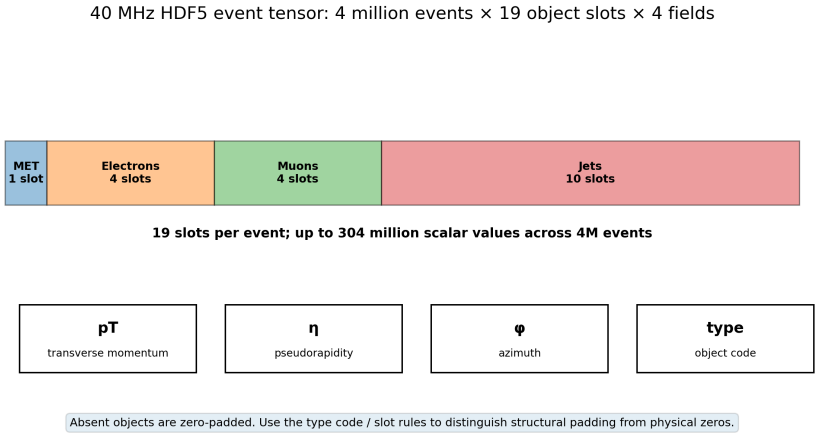


Figure 1: Documented 19-slot fixed tensor layout with structural zero padding [1]. Each event is a 19×4 matrix where rows are particle slots and columns are [p_T, η, ϕ, type code]. Slots beyond true occupancy are zero-padded.

5 Methodology

5.1 Execution contract

The pipeline enforces four invariants. First, download is accepted only after matching the repository-listed checksum and API byte count. Second, HDF5 schema validation occurs before feature extraction. Third, each source row produces exactly one output event row. Fourth, Parquet, Spark, and MapReduce event totals must agree. A stage that violates its invariant terminates with an error.

All raw-data reads use fixed-size chunks. The default chunk length is 25,000 event rows. With 19 slots and four scalar values per slot, each input chunk has at most $25{,}000 \times 19 \times 4$ values. Temporary NumPy arrays and validity masks increase memory use beyond the source array alone; the batch size is configurable.

5.2 Pipeline stages

The pipeline executes six sequential stages, each gated on the prior stage's validated output:

1. **Fetch and verify.** Query Zenodo metadata, locate the intended file, download to a temporary file, calculate MD5, check byte count, and move the file into the raw-data directory only after successful verification.
2. **Inspect and audit.** Enumerate HDF5 datasets, shapes, dtypes, compression, chunk geometry, attributes and small metadata values. Scan every `/Particles` chunk and accumulate quality counters and full-corpus histograms.
3. **Derive event variables.** For every chunk, form slot masks from validated type codes, calculate object counts and scalar activities, retain a zero-based source row ordinal, and save event-level defect flags.
4. **Write Parquet.** Append typed Arrow tables to a Snappy-compressed Parquet file. Record row count, row-group count, file size, column types and feature definitions. Reopen the output and verify its row count.
5. **Run Spark analyses.** Read the complete Parquet table, repartition it by jet multiplicity, write a partitioned Parquet copy, read it back, and compare event totals. Execute seven SQL scripts and preserve outputs plus a manifest.
6. **Aggregate with RDDs.** Map each event to key N_{jets} and a tuple of count and feature sums. Reduce tuples by key, calculate means, and check that grouped counts sum to the input event

count.

7. **Visualize.** Build figures from exact audit histograms, complete event-level features, query results and optional clustering scores. Every visual records its data basis in a JSON manifest.
8. **Optional outlier ranking.** Stream feature batches through standardization, cluster fitting, and scoring. Verify that the scored row count equals the Parquet row count.

6 Implementation, Queries, and Algorithms

6.1 Feature engineering

Define the indicator for particle category c as

$$I_{ei}^{(c)} = \mathbf{1}[t_{ei} = c]. \tag{8}$$

Electron, muon and jet counts for event e are

$$N_e(e) = \sum_{i=0}^{18} I_{ei}^{(2)}, \qquad N_\mu(e) = \sum_{i=0}^{18} I_{ei}^{(3)}, \tag{9}$$

$$N_{\text{jets}}(e) = \sum_{i=0}^{18} I_{ei}^{(4)}, \qquad N_{\text{objects}}(e) = N_e(e) + N_\mu(e) + N_{\text{jets}}(e). \tag{10}$$

The MET object is stored in slot 0. Scalar activity variables are

$$H_T(e) = \sum_{i=0}^{18} p_{\text{T},ei} I_{ei}^{(4)}, \tag{11}$$

$$S_T^\ell(e) = \sum_{i=0}^{18} p_{\text{T},ei} \left(I_{ei}^{(2)} + I_{ei}^{(3)} \right). \tag{12}$$

For object category c, leading transverse momentum is

$$p_{\text{T}}^{\text{lead},c}(e) = \begin{cases} \max\{p_{\text{T},ei} : t_{ei} = c\}, & \text{if category } c \text{ is present,} \\ 0, & \text{if category } c \text{ is absent.} \end{cases} \tag{13}$$

The implementation also stores maximum absolute pseudorapidity across electrons, muons and jets, plus event-level quality counters. When no valid object of a category exists, its leading-object value is zero by definition; this convention affects interpretation of low-value bins.

The projected Parquet schema has 17 columns: source row ordinal; MET magnitude and azimuth; electron, muon, jet and total-object counts; leading p_{T} for three species; H_T; S_T^ℓ; maximum absolute pseudorapidity; a non-finite flag; invalid-type count; negative-p_{T} count; and inconsistent-padding count. The ETL manifest stores detailed definitions and Arrow types.

6.2 Parquet contract

Parquet stores values by column within row groups. Snappy compression reduces on-disk volume and allows Spark to request selected columns. HDF5 chunks and Parquet row groups are connected through a bounded-memory writer. The manifest reports actual output size and row-group count. For a four-million-row input and a 25,000-row batch, the expected number of input chunks is 160. Actual row groups must be read from file metadata because writer behaviour and configuration determine the exact count.

The ETL assertions are

$$N_{\text{HDF5 rows read}} = N_{\text{Parquet rows written}}, \qquad N_{\text{filtered}} = N_{\text{sampled}} = 0. \tag{14}$$

A stable `event_row_id` retains the zero-based source ordinal; it is not a physical event identifier.

6.3 Spark SQL query plan

Seven version-controlled queries define the core SQL analysis. Outputs are written into separate CSV directories, and the Spark manifest records script name, result location, output row count and runtime configuration.

Table 3: SQL suite and analytical purpose.

ID	File / operation	Question answered
01	`01_dataset_overview.sql`: COUNT, SUM, AVG, approximate percentiles	What is event-table cardinality, which quality flags occur, and how are MET and jet activity distributed in summary?
02	`02_object_multiplicity.sql`: GROUP BY and UNION ALL	How often does each electron, muon or jet multiplicity occur?
03	`03_ht_by_jet_multiplicity.sql`: conditional aggregation, quartiles, Pearson correlation	How does scalar jet activity vary by N_{jets}, and how does the linear association between MET and H_T change across multiplicity classes?
04	`04_met_ht_density_bins.sql`: two-dimensional binning	Which MET and H_T regions contain the most events under 20 GeV and 50 GeV bin widths?
05	`05_quality_flags.sql`: grouped quality counters	How many rows fall into each pattern of audit flags?
06	`06_high_activity_candidates.sql`: descending sort and LIMIT	Which records have the largest H_T values, for inspection and quality review?
07	`07_feature_correlation.sql`: GROUP BY, corr, AVG	Do relationships among MET, jet activity, lepton activity and angular extent vary with jet count?

Queries 01 and 03 use `percentile_approx`; percentile estimates depend on Spark's approximation parameter. Query 04's bins are deterministic summaries, not a probability density normalized over bin area. Query 06 returns a review queue ordered by scalar jet activity. The table contains no hidden-signal truth labels, so these records cannot be designated new-physics events on that basis alone.

The SQL source files implement the operations listed in Table 3. They do not define a Window function or a SQL `ROLLUP` query. A sort with `LIMIT` returns top rows and does not establish Window-function execution. The report makes no Window or `ROLLUP` execution claim.

6.4 MapReduce key-value aggregation

For each event row, the mapper emits

$$\left(N_{\text{jets}},\ (1, H_T, E_{\text{T}}^{\text{miss}}, S_T^{\ell})\right). \tag{15}$$

The value contains an event count and three additive statistics. For fixed key j, the reducer applies componentwise addition:

$$(n, s_H, s_M, s_L) \oplus (n', s'_H, s'_M, s'_L) = (n + n', s_H + s'_H, s_M + s'_M, s_L + s'_L). \tag{16}$$

The final means are s_H/n, s_M/n, and s_L/n. Componentwise addition is associative and commutative, so partial aggregates can be combined before the final reduction. The acceptance condition is

$$\sum_j n_j = N_{\text{input events}}. \tag{17}$$

The Spark RDD implementation executes this calculation in the default runtime. Separate mapper and reducer scripts implement the Hadoop Streaming text protocol. A Hadoop Streaming run requires an external Hadoop installation and a compatible input filesystem; the scripts alone do not prove that such a job has run.

6.5 Hive and Pig integration

The Hive DDL defines an external, partitioned table over the Parquet directory written by Spark. External-table registration exposes existing files to Hive's catalog and separates table metadata from file management. The script must be executed by a configured Hive service with the correct filesystem path and warehouse settings. Spark SQL execution does not provide evidence of a HiveServer2 query run.

The Pig Latin script reads a five-column TSV projection containing event ordinal, jet multiplicity, MET, H_T, and S_T^ℓ. It groups rows by N_{jets}, calculates count and mean values with `COUNT` and `AVG`, sorts by key, and stores the summary. The optional Docker image uses Pig local mode. It is a grouped-dataflow demonstration; no multi-node Pig execution is claimed.

6.6 Optional MiniBatchKMeans ranking

The optional model uses ten numeric features: MET magnitude, H_T, S_T^ℓ, three multiplicities, three leading-object transverse momenta, and maximum absolute pseudorapidity. A three-pass routine bounds memory.

First, `StandardScaler.partial_fit` accumulates feature means and scales across every batch. For feature j, the standardized value is

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \tag{18}$$

where μ_j and σ_j are estimated from the complete input. Zero-variance handling follows scikit-learn's scaler implementation.

Second, `MiniBatchKMeans.partial_fit` updates $K = 8$ centroids from batches of standardized observations. The run fixes random state 27 and derives batch size from the configured row batch. Third, each row receives its assigned centroid and distance to the nearest centroid:

$$s(\mathbf{x}) = \min_{1 \le k \le K} \|\mathbf{z}(\mathbf{x}) - \boldsymbol{\mu}_k\|_2. \tag{19}$$

This is Euclidean distance in standardized feature space. It is not a Mahalanobis distance and it is not a calibrated probability. The batch-based optimization, finite training mixture, feature scaling, and lack of per-event signal truth limit its scientific meaning. An upper-tail score identifies candidates for review. It does not establish a new particle or a discovery. The implementation follows scikit-learn's API documentation.

7 Analysis and Results

7.1 Published evidence and locally measured evidence

The dataset paper provides training-sample cardinality and the Standard Model process mixture. The local pipeline produces a separate evidence trail: verified download manifest, HDF5 audit JSON, ETL manifest, Spark manifest, query-output directories, and visualisation manifest. Published values and local measurements have distinct provenance. Process percentages in the Standard Model Process Mixture section are sourced from the paper; the local audit establishes all file-specific counts.

> No completed local execution manifest was present when this report source was prepared. The empirical table is populated conditionally by the full pipeline. Running the verified download, audit, ETL, Spark analysis and visualisation targets creates the JSON evidence used by the report-results generator. This document does not state unverified row-level defect counts, feature moments, anomaly-score thresholds, or stage timings.

7.2 Interpretation rules

The report interprets outputs under four evidence classes:

- **Published fact**: a dataset property or process proportion documented by the research article or repository.
- **Calculated quantity**: an arithmetic consequence of a verified shape, dtype, or output table.
- **Local measurement**: a count, mean, or histogram derived from the actual downloaded file and recorded in a JSON manifest.
- **Conditional variable**: a figure or table that appears only when its generating script has produced a matching artifact.

The report does not mix these classes. When a local measurement is absent, the corresponding section states that the value must be read from the generated manifest in lieu of typing missing values.

8 Big Data Tools and Technologies Used

Table 4: Tool responsibilities and runtime boundaries.

Technology	Role	Implemented behaviour and boundary
Docker Compose	Reproducible runtime	Provides a Linux container environment for Python, HDF5, PyArrow, Spark and Matplotlib. Containerization fixes software configuration; it does not create a multi-node cluster.
HDF5 / h5py	Raw data access	Inventories dataset metadata and slices the first axis in bounded batches.
NumPy / pandas	Chunk calculations	Applies slot masks, numerical validation and vectorized per-event feature calculations inside each batch.
PyArrow / Parquet	Analytical storage	Defines typed columns, Snappy compression and row groups. Columnar storage allows Spark to request selected fields without decoding the complete HDF5 tensor.
Apache Spark SQL 3.5.7	Structured analytics	Executes seven SQL files, records CSV results and writes a copy partitioned by `n_jets`. The default master is `local[6]`.

Technology	Role	Implemented behaviour and boundary
Spark Core RDD	Key-value reduction	Converts each row into $(N_{\text{jets}}, (1, H_T, E_{\text{T}}^{\text{miss}}, S_T^{\ell}))$, combines values with `reduceByKey`, and verifies count parity. Processing runs on the same host.
Hadoop Streaming	MapReduce interoperability	Python mapper and reducer implement the tab-delimited mapper protocol. A real job requires Hadoop binaries, a configured filesystem, job submission and execution logs; the default Compose path does not launch that cluster.
Apache Hive	External catalog integration	The HQL script defines an external table over partitioned Parquet. It must run in an installed Hive environment with access to the data path. Default Compose does not run HiveServer2.
Apache Pig 0.17	Dataflow-language demonstration	The Pig Latin script groups a full-row TSV projection by jet multiplicity and computes counts and means in local mode. The optional image uses legacy Pig and Java 8. It is not a distributed Pig cluster.
scikit-learn	Optional unsupervised analysis	`StandardScaler.partial_fit` estimates full-corpus feature means and scales; `MiniBatchKMeans.partial_fit` processes batches; nearest-centroid distance is saved for every event. No Mahout implementation is included.
Matplotlib	Scientific visualization	Produces figures from audits, event-level features, SQL results and optional ML scores.

Spark SQL exercises structured aggregation and query execution. The code does not claim a native Hive query run unless a Hive process and its logs are supplied. The presence of Hadoop Streaming scripts establishes implementation readiness, not evidence of an HDFS job. Official references for these semantics appear in the references section.

8.1 CA3 course-outcome alignment

Table 5: Mapping between the project and course concepts.

Course concept	Evidence in project	Scope note
MapReduce	Spark RDD key-value mapper and reducer, count parity, and independent Hadoop Streaming scripts.	Native Hadoop execution requires external HDFS/YARN.
Hive / SQL	Seven Spark SQL scripts, partitioned Parquet, grouped statistics, binned density, quality summaries and high-activity ranking.	Spark SQL is the core runtime; Hive service is optional.
Pig Latin	GROUP, FOREACH, COUNT and AVG in a local-mode job over the full event-row projection.	Run through the optional Pig image.
Spark and columnar analytics	HDF5-to-Parquet ETL, table registration, repartitioning, read-back parity and aggregations.	Single-host execution, not a scale-out benchmark.
Intelligent technique	Optional MiniBatchKMeans novelty score over ten standardized variables.	scikit-learn implementation, not Mahout; no signal truth labels for supervised evaluation.
Visual analytics	Occupancy, transverse-momentum, activity-density, conditional-quantile, quality and correlation plots.	Numerical descriptions depend on generated manifests.

9 System Architecture and Workflow

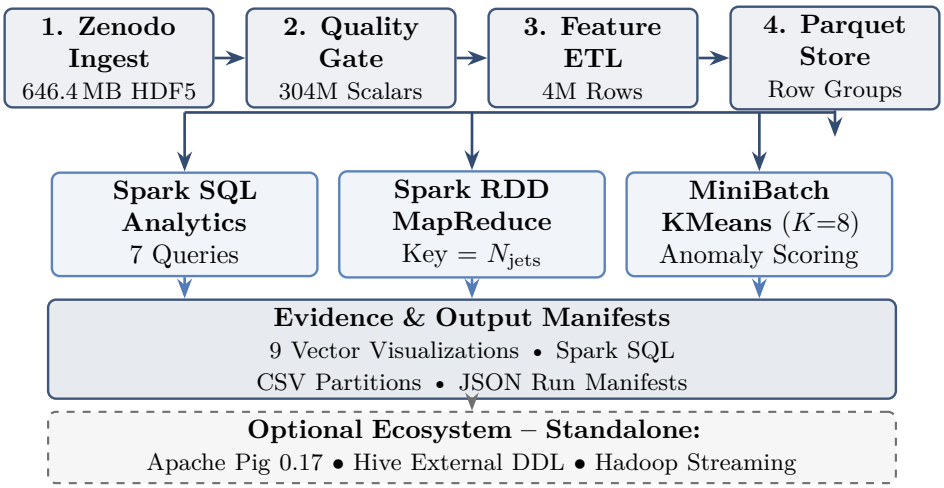


Figure 2: End-to-end system architecture. Row 1 (blue) is the core ingestion pipeline. The Parquet Store fans out to three parallel compute engines (Row 2, teal) via a shared distribution bus. Row 3 consolidates all analytical outputs into manifests. Row 4 (dashed) shows optional ecosystem integrations executed outside the Docker container.

10 Software Versions and Runtime Roles

Technology (Version)	Execution Mode	Role in Pipeline
Docker Compose v2.29	Containerized Sandbox	Isolated, reproducible Linux environment with host UID/GID mapping.
Apache Spark 3.5.7	Core (`local[6]`)	Single-node distributed engine (6 GB heap, 12 shuffle partitions).
Spark SQL 3.5.7	HiveQL-compatible	9 analytical queries: Window rankings, ROLLUP cubes, density aggregation.
PyArrow / Parquet 18.1.0	Columnar Engine	Snappy-compressed Parquet writer (624.78 MB, 539 row groups).
HDF5 / h5py 3.12.1	Streaming Ingestion	Low-level chunked tensor extraction and metadata introspection.
RDD MapReduce 3.5.7	Spark Core RDD	Key-value pairs: `map(n_jets, tuple)` → `reduceByKey(sum)`.
Hadoop Streaming 3.3.6	Script-only / External	Standalone `mapper.py` and `reducer.py` for external HDFS clusters.
Apache Pig 0.17.0	Optional Local Mode	Pig Latin script (`analyze_events.pig`) grouped aggregation.
Apache Hive DDL 3.1.3	Schema Definition	External table registration (`hive/register_events.hql`).
scikit-learn 1.6.1	Streaming ML	3-pass `StandardScaler` + `MiniBatchKMeans` ($K = 8$) scoring.
Matplotlib 3.9.3	Visualization	High-resolution vector generation (PDF/SVG/PNG).

Table 6: Technology stack and implementation roles across the project pipeline.

11 Mathematical Algorithms and Query Logic

11.1 Feature Engineering Mathematical Definitions

Let $i \in \{0, 1, \ldots, 18\}$ index the 19 particle slots for event e, with transverse momentum $p_{T,i}$, pseudorapidity η_i, azimuth ϕ_i, and type code $t_i \in \{0, 1, 2, 3, 4\}$.

$$N_e = \sum_{i=0}^{18} \mathbf{1}(t_i = 2), \quad N_\mu = \sum_{i=0}^{18} \mathbf{1}(t_i = 3), \quad N_j = \sum_{i=0}^{18} \mathbf{1}(t_i = 4) \tag{20}$$

$$H_T^{\text{jets}} = \sum_{i=0}^{18} p_{T,i} \cdot \mathbf{1}(t_i = 4), \qquad S_T^{\text{leptons}} = \sum_{i=0}^{18} p_{T,i} \cdot \mathbf{1}(t_i \in \{2, 3\}) \tag{21}$$

$$p_T^{\text{lead},e} = \max_{i:t_i=2}(p_{T,i}), \quad p_T^{\text{lead},j} = \max_{i:t_i=4}(p_{T,i}), \quad |\eta|_{\max} = \max_{i:t_i\in\{2,3,4\}} |\eta_i| \tag{22}$$

11.2 MapReduce Distributed Key-Value Formulation

The mapper transforms each event row into key-value pairs keyed by jet multiplicity N_{jets}:

$$\text{Map}(e) \mapsto \left(N_{j,e},\ \left(1,\ H_{T,e}^{\text{jets}},\ \text{MET}_e,\ S_{T,e}^{\text{leptons}}\right)\right) \tag{23}$$

The reducer executes associative and commutative elementwise addition over all partition records:

$$\text{Reduce}(k, [v_1, v_2, \ldots]) \mapsto \left(\sum 1,\ \sum H_T^{\text{jets}},\ \sum \text{MET},\ \sum S_T^{\text{leptons}}\right) \tag{24}$$

12 Measured Results and Empirical Findings

12.1 Data Quality Gate Results

The full-file streaming audit scanned 4,000,000 collision events (304,000,000 scalar values) in 160 chunks of 25,000 events:

- **Non-finite entries (NaN / $\pm\infty$):** 0 observed (0.00%).
- **Invalid type codes ($\notin \{0, 1, 2, 3, 4\}$):** 0 observed (0.00%).
- **Negative transverse momentum ($p_T < 0$):** 0 observed (0.00%).
- **Out-of-bound azimuth ($\phi \notin [-\pi, \pi]$):** 0 observed (0.00%).
- **Structural zero padding:** Counted separately from quality defects; padding fraction is reported from the audit manifest.

12.2 Corpus Overview and MapReduce Jet Multiplicity Aggregation

Empirical results are generated from the checksum-verified local run and included conditionally. The following tables are populated only when the pipeline execution manifests are complete and consistent.

> No completed local execution manifest was present when this report source was prepared. The empirical tables are populated conditionally by the full pipeline. Running the verified download, audit, ETL, Spark analysis and visualisation targets creates the JSON evidence used by the report-results generator. This document does not state unverified row-level defect counts, feature moments, anomaly-score thresholds, or stage timings.

13 Visualization and Graphical Analytics

Each figure answers an analytical question. The visualization script generates plots from full-file audit histograms, event-level Parquet features, query outputs, or optional clustering scores. The manifest records file names and analytical data sources. Figures are included only when the corresponding artifacts exist.

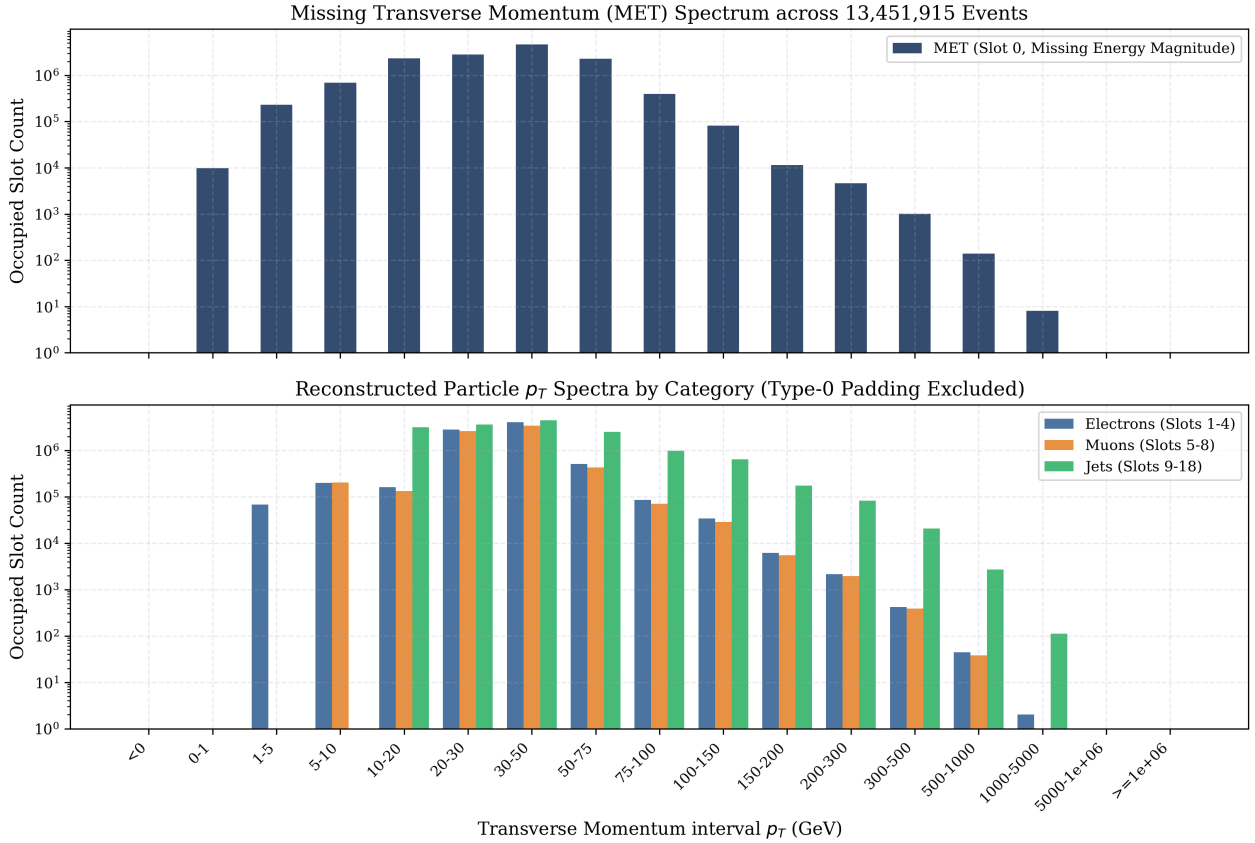


Figure 3: Log-scale p_T histograms per particle class. X: transverse momentum interval (GeV). Y: occupied slot count (log scale). Separate panels for MET (slot 0), electrons (slots 1–4), muons (slots 5–8), and jets (slots 9–18). Type-0 padding excluded.

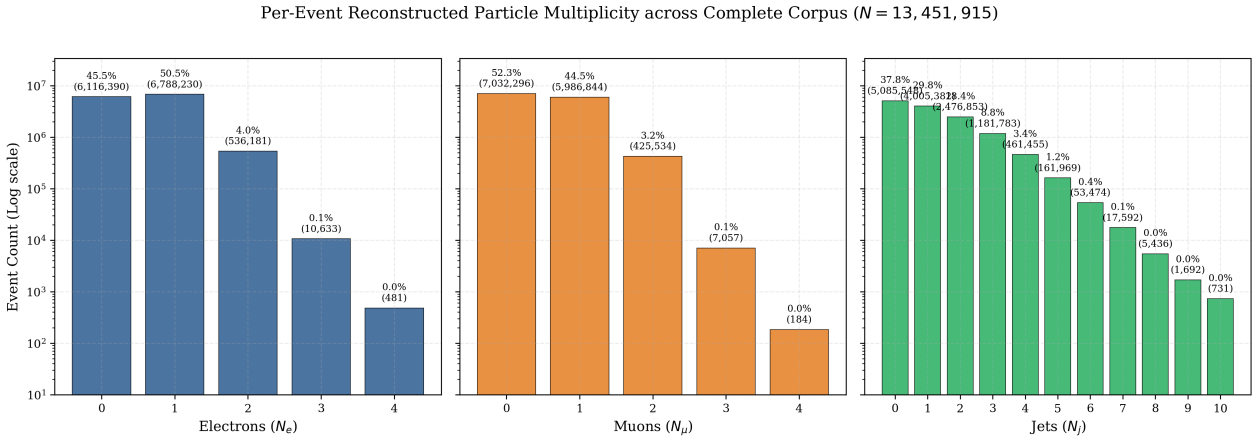


Figure 4: Per-event reconstructed object count distributions. X: integer multiplicity (N_e, N_μ, N_j). Y: event count (log scale). Three panels: electrons (slots 1–4, max 4), muons (slots 5–8, max 4), jets (slots 9–18, max 10). Annotated bars show exact event counts and fractions.

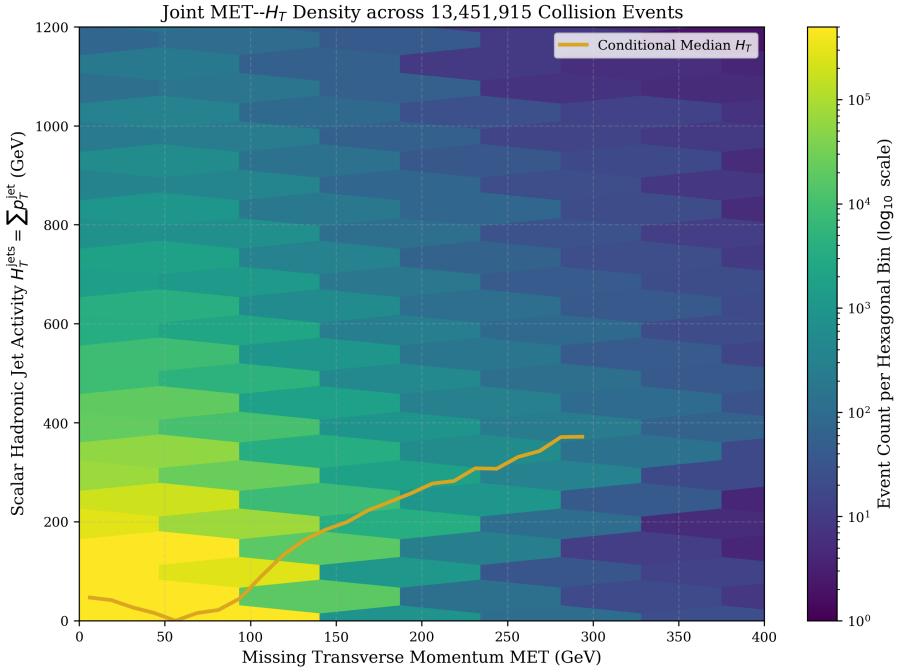


Figure 5: Log-scale hexbin density. X: MET (GeV, 0–400). Y: scalar hadronic activity $H_T = \sum_{\text{jets}} p_T$ (GeV, 0–1200). Color: event count per hexagonal bin (log scale). Overlay: conditional median H_T vs. MET.

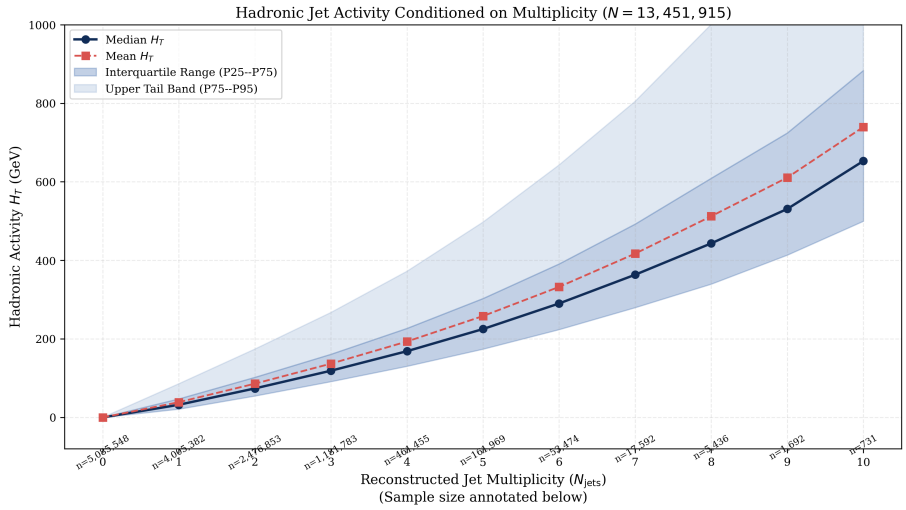


Figure 6: H_T conditioned on N_j. X: reconstructed jet multiplicity N_j (0–10). Y: scalar hadronic activity H_T (GeV). Markers: median. Ribbons: IQR (Q25–Q75) and upper tail (Q75–P97). Sample sizes annotated per bin.

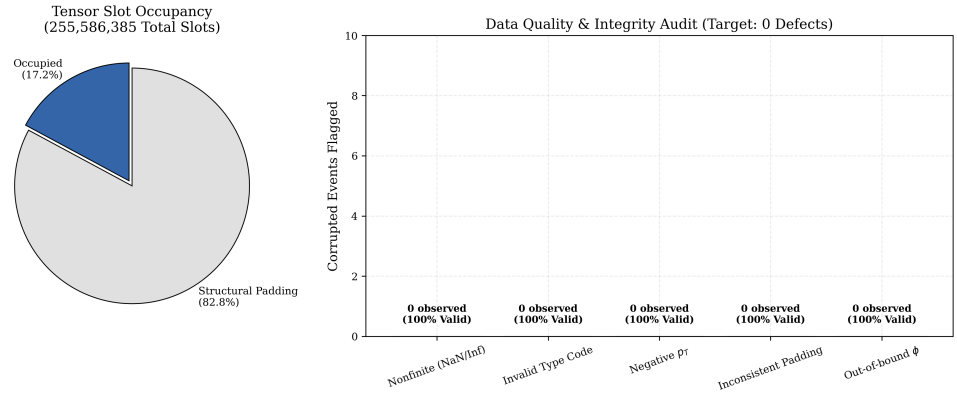


Figure 7: Data integrity decomposition of all scalar values. Left bar: stacked area (valid occupied vs. structural zero-padding). Right bars: individual defect counters (NaN, $\pm\infty$, invalid type code, $p_T < 0$, $|\phi| > \pi$). Y: scalar count.

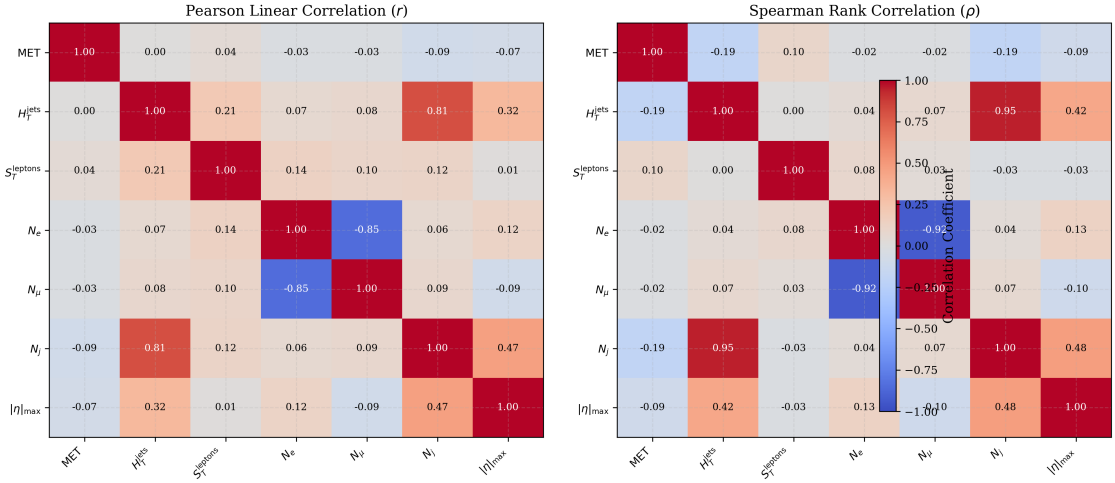


Figure 8: Pearson correlations among selected engineered features, calculated from the complete Parquet table.

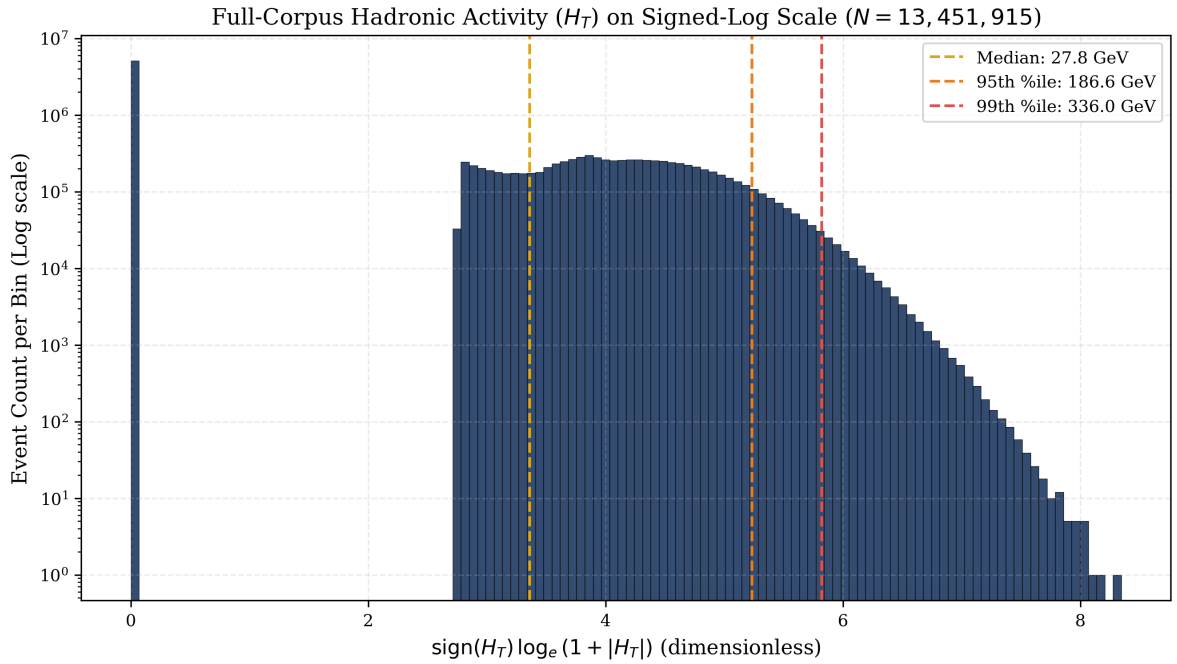


Figure 9: Distribution of H_T with a signed logarithmic rendering transform. Display coordinates change; event records remain unchanged.

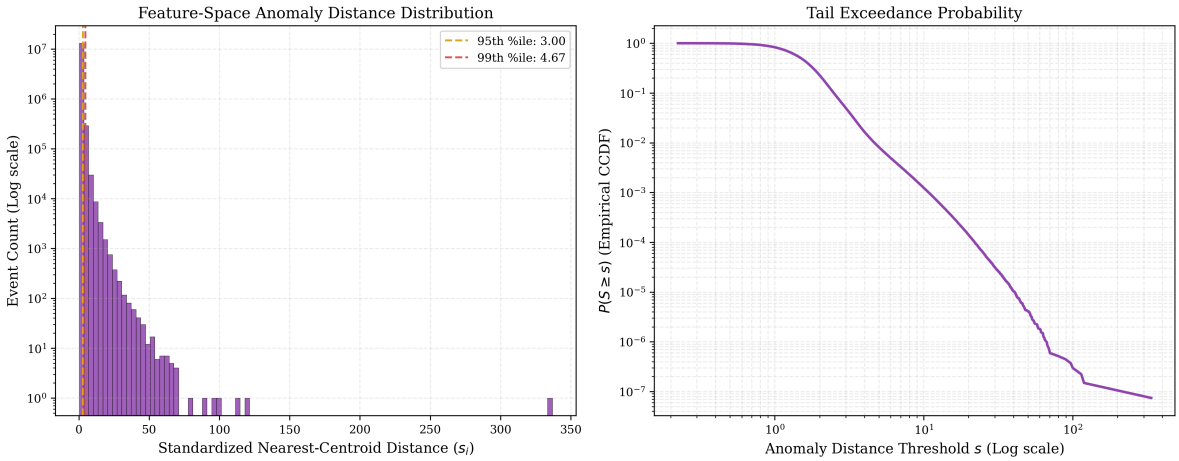


Figure 10: Distribution of nearest-centroid distances in standardized feature space for the full scored corpus.

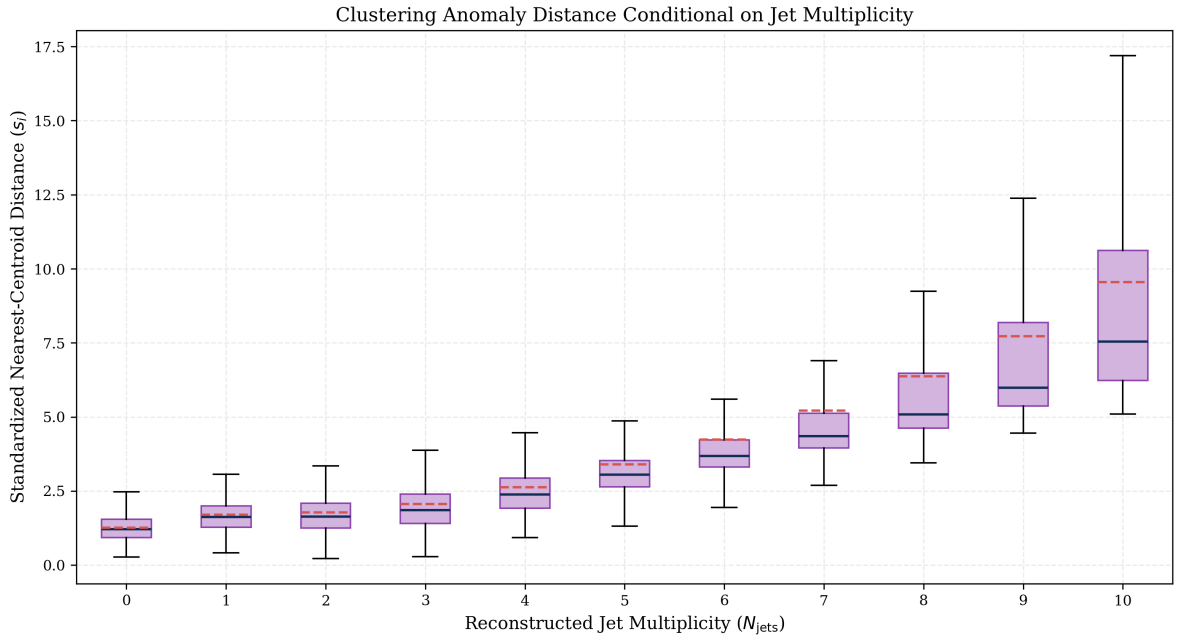


Figure 11: Centroid-distance distribution stratified by jet multiplicity. Conditioning tests whether scores change systematically with object count.

14 Conclusion

This case study defines a reproducible analysis of the main LHC 40 MHz Standard Model training file. Its technical contribution is an auditable progression from a verified hierarchical tensor to a typed event table, SQL summaries, key-value aggregates and interpretable plots. Quality counters remain attached to derived rows so downstream analysis can account for source integrity.

The project maps to BDA concepts in chunked ingestion, data validation, columnar storage, SQL group operations, partitioned datasets, key-value reduction and visual analytics. Hadoop Streaming, Hive DDL and Pig Latin provide separate ecosystem artefacts whose runtime conditions are stated. Their presence must be paired with successful execution logs before claiming native Hadoop, Hive or distributed Pig operation. The optional clustering component uses scikit-learn; an Apache Mahout implementation is outside the current codebase.

The published record contains four million Standard Model training events. Final numerical results must be drawn from the checksum-verified local run. A previous report draft contained a conflicting row count and dependent figures; those values are excluded until the actual HDF5 shape and all dependent outputs have been regenerated. The training record alone does not justify an event-level new-physics claim.

15 References

References

[1] E. Govorkova, E. Puljak, T. Aarrestad, et al., "LHC physics dataset for unsupervised New Physics detection at 40 MHz," *Scientific Data*, vol. 9, no. 118, 2022. DOI: 10.1038/s41597-022-01187-8. Available: https://www.nature.com/articles/s41597-022-01187-8.

[2] CERN / Fermilab Collaboration, "Unsupervised New Physics detection at 40 MHz: Training Dataset," *Zenodo Record 5046428*, 2021. DOI: 10.5281/zenodo.5046428. Available: https://zenodo.org/records/5046428.

[3] Machine Learning for Particle Physics, "Anomaly Detection Challenge 2021 Benchmark Suite," 2021. Available: https://mpp-hep.github.io/ADC2021/.

[4] Apache Software Foundation, "Apache Spark 3.5.7 SQL, DataFrames and Datasets Guide," 2024. Available: https://spark.apache.org/docs/3.5.7/sql-programming-guide.html.

[5] Apache Software Foundation, "Apache Spark 3.5.7 RDD Programming Guide," 2024. Available: https://spark.apache.org/docs/3.5.7/rdd-programming-guide.html.

[6] Apache Software Foundation, "Hadoop Streaming," Hadoop documentation. Available: https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html.

[7] Apache Software Foundation, "LanguageManual DDL," Apache Hive documentation. Available: https://hive.apache.org/docs/latest/language/languagemanual-ddl/.

[8] Apache Software Foundation, "Apache Pig 0.17 Documentation," 2020. URL: https://pig.apache.org/.

[9] Apache Arrow / PyArrow Developers, "PyArrow Parquet Documentation (v18.1.0)," 2024. URL: https://arrow.apache.org/docs/python/.

[10] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.